@DevFest

Mauritius

2026

The Human Is the Reviewer: Shipping a Production Multi-Agent Content Platform on Gemini and Cloud Run

Sangalo Mwenyinyo
CTO @ Prescott Data · GDE, Modern & Enterprise Infrastructure

Google Developer Groups

One business. Seven platforms. One person doing it all by hand.

- Instagram, Facebook, YouTube, X, TikTok, LinkedIn, your website, your newsletter
- Each one wants a different format: a reel, a carousel, a thread, a 1,200-word post
- So you write it again. And again. By hand, platform by platform
- Most days the content calendar wins and the business loses



Google Developer Groups

Make it once. Syndicate everywhere.

- One source: a YouTube video, an article, a webpage or a GitHub repo
- Agents turn it into a blog draft, a cover image, a carousel and platform-native posts
- Every piece lands in a review queue. You approve. Then it ships
- Humans don't write anymore. They review.

Demo: ContentForge



Google Developer Groups

Three choices that made this possible

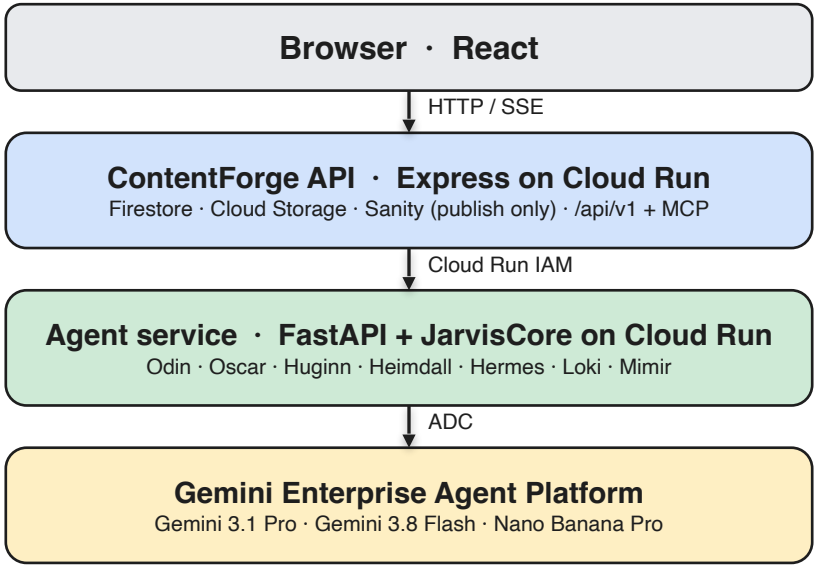
Layer	What we used	Why
Agent runtime	jarviscore	Purpose-built for cooperating agents
Models	Gemini 3.1 Pro · Gemini 3.8 Flash · Nano Banana Pro	The right model for each job
Infrastructure	Cloud Run, serverless	Scales to zero, IAM between services, no API keys

Models served through Gemini Enterprise Agent Platform, authenticated with ADC



Google Developer Groups

Three services, one review queue



Firestore is the source of truth while content is in review.

Sanity, the CMS, is written once, at publish time, only from approved.

Machines use the same API over REST or MCP with cfk_ keys that can never approve.

Google Developer Groups

JarvisCore: an agent framework and runtime

- Open source, Python, built by Prescott Data
- You write a role, a capability and a handler. The runtime does the rest
- P2P mesh: agents discover peers by role and delegate with one call
- Lifecycle, logging and tracing built in, so FastAPI only owns HTTP
- CustomAgent for control, AutoAgent when the kernel should plan

Star it on GitHub

github.com/Prescott-Data/jarviscore-framework

Google Developer Groups

One prompt did four jobs. It did all of them averagely.

- Writing, fact-checking, editing and designing are different skills
- One prompt hallucinated numbers, buried the lede and drifted off brand
- One agent per concern: one prompt, one model tier, one failure policy each
- If research or QA fails, you still get the draft, flagged, in the review queue
- Specialists are easier to test, swap and upgrade



Google Developer Groups

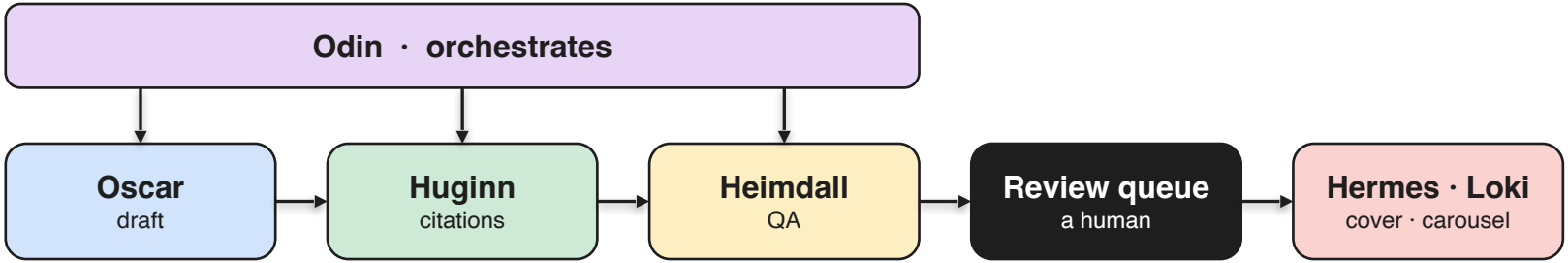
Seven agents. One job each. The right model for each job.

Agent	Job	Model	Why this model
Odin	Orchestrates the pipeline, art-directs the cover	Gemini 3.1 Pro	Judgement calls and tool use
Oscar	Writes the long-form draft	Gemini 3.1 Pro	Long context, strong prose
Huginn	Researches claims, inserts citations	Gemini 3.8 Flash	Many small, fast calls
Heimdall	Editorial QA: tone, fidelity, cuts	Gemini 3.1 Pro	The last reader before a human
Hermes	Generates the cover image	Nano Banana Pro	Clean text rendering in images
Loki	Social copy, carousel slots, studio chat	3.8 Flash → 3.1 Pro fallback	Short, frequent, streamed to a person
Mimir	Scouts the Source Vault for banked material	Gemini 3.8 Flash	Summarises in batches of ten



Google Developer Groups

One transcript, four hand-offs, one review queue



- Oscar is the only hard dependency: no draft, no pipeline
- Everything after Oscar fails open: a research or QA failure keeps the last good draft and flags the stage
- Each hand-off streams a stage event to the browser: oscar_done · huginn_none · heimdall_done
- This morning's run: 109 seconds and $0.16 from transcript to reviewed draft

Google Developer Groups

Model IDs live in config, resolved on every call

```
def _route(m):                    # bare name → Gemini Enterprise Agent Platform via ADC
    return m if "/" in m else f"vertex_ai/{m}"

GEMINI_MODEL       = gemini-3.1-pro-preview   # Odin, Oscar, Heimdall
GEMINI_FLASH_MODEL = gemini-3.8-flash         # Loki, Huginn, Mimir
IMAGE_MODEL        = gemini-3-pro-image       # Hermes
HEIMDALL_MODEL, LOKI_MODEL, ...               # per-agent override, same shape
```

- Swap a model with an environment change on the Cloud Run revision, no redeploy
- Pro where judgement matters, Flash where volume matters
- The same code ran a different vendor mix last week. Today it's all Gemini



Google Developer Groups

The model chooses the words. Code chooses everything else.

```
# What Loki may return — nothing else
{"slides": [{"kind": "hook", "heading": "...", "body": "..."}, ...]}

# carousel_pdf.py — brand facts live in code
BRAND_THEMES["your_brand"] = {
    "bg": "#F5F3EE", "accent": "#2E7DF6",
    "heading_font": "Fraunces", "masthead": "YOUR BRAND"}
CANVAS_W, CANVAS_H = 1080, 1350   # deterministic HTML → PDF
```

- Upgrade the model: the carousel looks identical
- Change the brand: edit a dict, every future carousel follows
- Every string is escaped on the way in: a transcript can change what a slide says, not what a slide is



Google Developer Groups

Four things to adapt to your brand

- Split by concern: one agent, one prompt, one model tier, one failure policy
- Fail open after the draft. Flag, don't discard
- Retries for the start of a call, failover for the middle, and make it loud
- The model fills slots. Code owns the design and the publish button



Google Developer Groups

Questions?

Sangalo Mwenyinyo

github.com/sangalo20

linkedin.com/in/sangalo-mwenyinyo-b3803b192

medium.com/@mwenyinyo

substack.com/@sangalom

Star it on GitHub

github.com/Prescott-Data/jarviscore-framework

Google Developer Groups